IIoT Communication Protocols — A Practical Guide

A working engineer's reference to the protocols used to move data around industrial systems: from a sensor on a machine, up through controllers and gateways, to historians, MES and cloud analytics.

These guides are **originally written** for this repository. They do not reproduce any standards body's copyrighted specification text — see `references.md` for where to obtain the official documents.

Contents

#	Guide	Covers
01	Landscape & architecture	OT vs IT, the automation pyramid, Unified Namespace, determinism classes, pub/sub vs request/response
02	MQTT & Sparkplug B	Topics, QoS, retain, LWT, MQTT 5 features, Sparkplug payloads and state management
03	OPC UA	Information modelling, address space, services, PubSub, security, profiles
04	Modbus	RTU/ASCII/TCP framing, the four register tables, function codes, real-world pitfalls
05	Industrial Ethernet	PROFINET, EtherNet/IP/CIP, EtherCAT, POWERLINK, Sercos III, conformance classes
06	Fieldbus & device level	PROFIBUS, HART, Foundation Fieldbus, CANopen, J1939, IO-Link, AS-Interface
07	CoAP, AMQP & DDS	Constrained REST, enterprise messaging, and real-time data-centric middleware
08	Wireless & LPWAN	WirelessHART, ISA100, Zigbee, BLE, LoRaWAN, NB-IoT/LTE-M, 5G URLLC
09	Security	IEC 62443, Purdue zones, TLS and PKI in OT, per-protocol security posture
10	Integration patterns	Gateways, protocol conversion, edge buffering, store-and-forward, UNS rollout
—	References	Official specifications, standards bodies, licensing notes

Quick pick

"I need to get PLC data to the cloud." OPC UA for the semantic model at the plant, MQTT + Sparkplug B for the transport north of the gateway. See 10.

"I need to read a flow meter over two wires." Modbus RTU if it is a simple register device; HART if it is a 4–20 mA transmitter you also want diagnostics from. See 04 and 06.

"I need motion control with microsecond jitter." EtherCAT or POWERLINK, or PROFINET IRT. Not MQTT, not OPC UA client/server. See 05.

"I need battery sensors reporting a few times an hour over kilometres." LoRaWAN or NB-IoT. See 08.

"I need robot/vehicle software components to share data at high rate." DDS. See 07.

Comparison at a glance

Protocol	Model	Typical cycle	Transport	Discovery	Built-in security	Payload is self-describing
Modbus TCP	Polled register read/write	10 ms – 1 s	TCP/502	None	None	No
Modbus RTU	Polled register read/write	20 ms – 1 s	RS-485	None	None	No
OPC UA client/server	Session + subscriptions	50 ms – 1 s	TCP/4840, HTTPS	LDS / GDS	Yes (X.509, per-message)	Yes
OPC UA PubSub	Publish/subscribe	1 – 100 ms	UDP multicast, MQTT, AMQP	Yes	Yes	Yes
MQTT	Broker pub/sub	Event-driven	TCP/1883, TLS/8883	None (topics by convention)	TLS + authN	No (bytes)
Sparkplug B	Broker pub/sub + state	Event-driven	MQTT	Yes (birth certificates)	Inherits MQTT	Yes (protobuf, typed)
CoAP	REST on UDP	Event or poll	UDP/5683, DTLS/5684	CoRE Link Format	DTLS / OSCORE	Partly (CBOR/SenML)
AMQP 1.0	Brokered queues/topics	Event-driven	TCP/5672, TLS/5671	None	TLS + SASL	Yes (typed)
DDS / RTPS	Data-centric pub/sub	0.1 – 10 ms	UDP multicast	Automatic (SPDP/SEDP)	DDS Security	Yes (IDL types)
PROFINET RT	Cyclic I/O	1 – 10 ms	Raw Ethernet 0x8892	DCP	Limited	Via GSDML
PROFINET IRT	Scheduled cyclic I/O	31.25 µs – 1 ms	Raw Ethernet + scheduling	DCP	Limited	Via GSDML
EtherNet/IP	Implicit (I/O) + explicit	1 – 10 ms	UDP/2222, TCP/44818	ListIdentity	CIP Security	Via EDS
EtherCAT	Summation frame	50 µs – 1 ms	Raw Ethernet 0x88A4	Automatic	FSoE for safety only	Via ESI
LoRaWAN	Uplink/downlink via gateway	Minutes – hours	Sub-GHz radio	Join procedure	AES-128 (two keys)	No
HART	Digital over 4–20 mA	~2 updates/s	Current loop	Yes	None (WirelessHART: AES)	Yes (device description)

How to read these guides

Each protocol guide follows the same shape so you can compare them directly:

1. **What it is and where it sits** — the layer and the problem it solves
2. **How it works** — framing, addressing, data model
3. **Configuration that matters** — the handful of settings that cause most trouble
4. **Security posture** — what it does and does not protect
5. **When to use it / when not to**
6. **Pitfalls** — the failures that actually happen in the field

Scope and honest limits

These are **integration-oriented** guides. They are written to help you choose a protocol, wire it up correctly, and avoid common failure modes. They are not a substitute for the normative specification when you are *implementing* a stack or certifying a device — for that you need the official document, and `references.md` tells you where each one lives and what it costs.

Figures such as cycle times are typical field values, not guaranteed limits; the achievable number always depends on the specific hardware, topology and configuration.

01 — Landscape & Architecture

Why industrial networks look strange to IT engineers

An industrial network optimises for different things than an office network. A web service cares about throughput and average latency. A machine controller cares about **determinism**: the guarantee that a message arrives within a bounded time, every time. A control loop that usually completes in 1 ms but occasionally takes 40 ms is not a slow control loop — it is a broken one, and it may wreck the product or the machine.

This single difference explains most of the design decisions that follow: why fieldbuses avoid TCP, why some protocols bypass the IP stack entirely, and why "just put it all on MQTT" is not an answer at the control layer.

The automation pyramid

The traditional model stacks five levels. It is dated — but it is still the mental map most plant engineers use, and the vocabulary is unavoidable.

```
Level 4   ERP / business systems      SAP, Oracle          days - months
          ------------------------------------------------------------
Level 3   MES / operations            scheduling, batch    minutes - hours
          ------------------------------------------------------------
Level 2   SCADA / HMI                 operator screens     100 ms - 1 s
          ------------------------------------------------------------
Level 1   Control: PLC / DCS / PAC    the control logic    1 - 100 ms
          ------------------------------------------------------------
Level 0   Field: sensors, drives      the physical process 10 us - 10 ms
```

Protocols cluster by level:

- **Level 0-1**: EtherCAT, PROFINET, EtherNet/IP, PROFIBUS, IO-Link, HART
- **Level 1-2**: Modbus TCP, OPC UA client/server, OPC Classic (DA)
- **Level 2-3**: OPC UA, MQTT/Sparkplug, SQL historians, REST
- **Level 3-4**: AMQP, Kafka, REST, file transfer

The problem with the pyramid

Data has to be relayed up level by level, and each hop is a hand-built mapping: a PLC tag becomes a Modbus register number, which a SCADA system maps to a point name, which an MES integration maps to a database column. Change a tag and three layers break. This is the "integration tax" that dominates the cost of most industrial data projects.

The Unified Namespace alternative

The Unified Namespace (UNS) inverts the flow. Instead of each layer polling the one below, every producer **publishes its current state** to a central broker using a hierarchical topic that mirrors the physical plant, and any consumer subscribes to whatever it needs.

```
enterprise/site/area/line/cell/device/metric

acme/dallas/packaging/line3/filler/motor1/temperature
```

Properties worth understanding:

- **Report by exception.** A value is published when it changes, not on a poll cycle. Quiet plants generate almost no traffic.

- **Single source of current state.** With MQTT retained messages or Sparkplug birth certificates, a newly connected client gets the present value immediately without waiting for the next change.
- **Producers do not know consumers.** Adding an analytics tool requires no change to any PLC or gateway.
- **The broker becomes critical infrastructure.** It needs clustering, monitoring and a security model — this is the main cost of the pattern.

A UNS does *not* replace Level 0-1 protocols. Nobody closes a servo loop over MQTT. It replaces the brittle Level 2-4 integration layer.

Communication models

Request/response (polling)

A client asks, a server answers. Modbus and OPC UA reads work this way.

- Simple, easy to debug, client controls the load.
- Bandwidth is spent whether or not data changed.
- Latency to detect a change is up to one full poll cycle.
- Scaling is poor: N clients polling M devices is N x M traffic.

Publish/subscribe (brokered)

Publishers send to a topic on a broker; subscribers receive. MQTT and AMQP work this way.

- Decouples producers and consumers in space, time and quantity.
- Traffic proportional to *change*, not to the number of clients.
- The broker is a single point of failure and a bandwidth concentrator.
- Harder to reason about end-to-end timing.

Publish/subscribe (brokerless, data-centric)

Participants discover each other directly and exchange typed data. DDS works this way, as does OPC UA PubSub over UDP multicast.

- No broker hop, so lower and more predictable latency.
- Discovery traffic grows with participant count and needs multicast.
- Rich QoS contracts (deadline, liveliness, durability) are negotiated per topic.

Cyclic I/O exchange

Controller and device swap a fixed-size data image every cycle, forever, whether or not anything changed. PROFINET, EtherNet/IP implicit messaging and EtherCAT work this way.

- Jitter is bounded and tiny; this is what real-time control needs.
- Payloads are fixed at configuration time; there is no ad-hoc querying.
- Needs engineering tools and device description files (GSDML, EDS, ESI).

Determinism classes

A useful way to classify any industrial link:

Class	Cycle time	Jitter tolerance	Typical use	Technologies
Isochronous / motion	31.25 us - 1 ms	< 1 us	Coordinated multi-axis motion, CNC	EtherCAT DC, PROFINET IRT, POWERLINK, Sercos III
Hard real-time	1 - 10 ms	< 100 us	Standard machine I/O, safety	PROFINET RT, EtherNet/IP, Modbus RTU on dedicated link
Soft real-time	10 - 100 ms	milliseconds	Process control, SCADA polling	Modbus TCP, OPC UA subscriptions
Non-real-time	> 100 ms	seconds	Historians, dashboards, analytics, cloud	MQTT, AMQP, REST, Kafka

Most "IIoT" work lives in the bottom two rows. Recognising that early saves a lot of argument: if your use case is a dashboard refreshed every few seconds, you do not need — and should not pay for — a deterministic fieldbus.

Where the OSI model helps and where it misleads

Several industrial protocols deliberately skip layers to cut latency:

- **EtherCAT** rides directly on Ethernet with EtherType `0x88A4`. No IP, no TCP. You cannot route it; you cannot see it with a normal IP-based tool.
- **PROFINET RT** uses EtherType `0x8892` for cyclic data, while acyclic configuration and diagnostics use UDP/IP. The same cable carries both.
- **Modbus RTU** is effectively an application protocol sitting straight on a serial line, with no network layer at all.

Practical consequences: a router or L3 switch will drop EtherCAT and PROFINET RT frames. VLANs and managed switches must be configured with these EtherTypes in mind. And a packet capture filtered on `tcp` or `ip` will show you nothing of the actual control traffic.

Choosing: the three questions that decide it

1. **What is the required cycle time, and what jitter can the process tolerate?** This alone eliminates most options. Sub-millisecond with tight jitter forces a dedicated industrial Ethernet fieldbus.
1. **Who needs the data, and does that set change over time?** One fixed controller reading one device: polling is fine. An unknown and growing set of consumers: publish/subscribe will pay for itself.
1. **Does the consumer need to understand the data without a human explaining it?** If yes, you need a protocol carrying semantics — OPC UA, Sparkplug B or DDS — not raw Modbus registers. This is the difference between `40001 = 1247` and `Line3.Filler.Motor1.Temperature = 124.7 degC (good, 2026-10-20T08:14:22Z)`.

Pitfalls

- **Assuming Ethernet means interoperable.** PROFINET, EtherNet/IP and EtherCAT all run on Ethernet cable and are mutually unintelligible.
- **Polling a device faster than it updates.** Many transmitters refresh internally at 100-500 ms. Polling at 10 ms gives you the same value ten times and may starve the device's own processing.
- **Putting control traffic and video on one untagged VLAN.** A camera burst will blow the jitter budget of a cyclic bus.
- **Forgetting the gateway is a translator, not a pipe.** Every protocol conversion loses something: data types collapse, timestamps get replaced with gateway time, quality flags vanish. Know what you are losing at each hop.

- **Designing the topic namespace last.** In a UNS the namespace *is* the API. Rename it after a hundred devices are deployed and you will do it twice.

02 — MQTT & Sparkplug B

What it is and where it sits

MQTT is a lightweight publish/subscribe transport. A client connects to a **broker** over TCP, publishes messages to **topics**, and subscribes to topic filters. The broker does the routing; publishers and subscribers never address each other.

It sits at Level 2-4 — gateway to cloud, edge to historian. MQTT itself carries **opaque bytes** and defines no data model, which is both its great strength (it will carry anything) and its great weakness (nothing knows what the bytes mean). Sparkplug B, covered below, is the layer that fixes this for industrial use.

Versions you will meet: **3.1.1** (OASIS, 2014 — still the most widely deployed) and **5.0** (OASIS, 2019 — worth using for new work, see the feature list below).

How it works

Topics

Topics are UTF-8 strings with `/` as a hierarchy separator. There is no registration step; publishing to a topic creates it implicitly.

```
acme/dallas/packaging/line3/filler/motor1/temperature
```

Subscriptions may use two wildcards:

Wildcard	Meaning	Example	Matches
`+`	exactly one level	`acme/dallas/+/line3/#`	any area
`#`	all remaining levels (must be last)	`acme/dallas/#`	everything under the site

Rules worth knowing: topics are case-sensitive; a leading `/` creates an empty first level (`/acme` is not `acme`); `$SYS/...` is reserved by convention for broker statistics; and you cannot publish to a wildcard.

Quality of Service

QoS is negotiated per publish and per subscription; the effective QoS of delivery is the **minimum** of the two.

QoS	Guarantee	Handshake	Cost	Use for
0	At most once — fire and forget	PUBLISH	1 message	High-rate telemetry where a gap is harmless
1	At least once — **duplicates possible**	PUBLISH / PUBACK	2 messages	Most industrial telemetry
2	Exactly once	PUBLISH / PUBREC / PUBREL / PUBCOMP	4 messages	Commands, totalisers, billing counters

The practical advice: **QoS 1 for data, QoS 2 for commands**. QoS 1 duplicates are real and will appear after a reconnect — make consumers idempotent. QoS 2 costs two round trips and will throttle a high-rate stream.

Retained messages

A publish with the **retain** flag set causes the broker to store that one message as the "last known good" value for the topic and deliver it immediately to any future subscriber.

This is how you solve the cold-start problem: a dashboard that connects at 03:00 gets the current tank level at once, instead of a blank field until the level next changes. Exactly one retained message is kept per topic; publishing a zero-length payload with retain set clears it.

Last Will and Testament

At connect time a client may register a **will** message — topic, payload and QoS. If the client disconnects ungracefully (no DISCONNECT packet, or keepalive expiry), the broker publishes that message on its behalf.

This is the standard device-offline detection mechanism. Combine it with a retained birth message on the same topic and subscribers always know whether a device is alive:

```
Topic:   acme/dallas/line3/filler/status
Connect: publish "ONLINE"  (retain=true)
Will:    publish "OFFLINE" (retain=true)
```

Keepalive and sessions

`Keep Alive` is the maximum interval the client promises between packets; if it has nothing to say it sends PINGREQ. The broker declares the client dead after **1.5 x keepalive**. Short keepalives detect failure faster but cost traffic on cellular links.

Sessions determine what survives a disconnect. In MQTT 5 a session persists for `Session Expiry Interval` seconds, during which the broker holds the client's subscriptions and queued QoS 1/2 messages. In 3.1.1 the cruder `Clean Session` flag either keeps state indefinitely or discards it entirely.

MQTT 5 features that matter industrially

Feature	Why you care
Reason codes on every ack	A rejected publish tells you *why* instead of silently dropping the connection
Session Expiry Interval	Bounded state retention; fixes the 3.1.1 "persistent session forever" leak
Message Expiry Interval	Stale telemetry is discarded rather than delivered hours late after an outage
Topic Alias	Replaces a long topic with a 2-byte integer after first use — significant on metered cellular
User Properties	Arbitrary key/value headers; carry units, source, trace IDs without touching the payload
Shared Subscriptions (`$share/group/topic`)	Load-balance one topic across a pool of consumers — enables horizontal scaling
Request/Response (`Response Topic` , `Correlation Data`)	Standardised RPC pattern instead of hand-rolled reply topics
Flow control (`Receive Maximum`)	Prevents a fast publisher overwhelming a slow consumer

Sparkplug B

MQTT's silence about payloads means two vendors can both "support MQTT" and still not interoperate at all. **Sparkplug B** (an Eclipse Foundation specification) closes this by defining three things on top of MQTT:

1. **A topic namespace**
2. **A payload format** (Google Protocol Buffers, with typed metrics)
3. **A state management model** (birth and death certificates)

Topic structure

```
spBv1.0/<group_id>/<message_type>/<edge_node_id>[/<device_id>]
```

Message type	Meaning
`NBIRTH` / `NDEATH`	Edge node birth / death certificate
`DBIRTH` / `DDEATH`	Device birth / death certificate
`NDATA` / `DDATA`	Node / device data (changes only)
`NCMD` / `DCMD`	Command to node / device
`STATE`	Primary application state (host online/offline)

Why birth certificates are the key idea

On connect, an edge node publishes `NBIRTH` listing **every metric it will ever report**, with name, datatype, an integer alias and the current value. Subsequent `NDATA` messages carry only what changed, referencing the compact alias instead of the full name.

The consequences are worth spelling out, because this is what makes Sparkplug genuinely different from ad-hoc MQTT:

- **Auto-discovery.** A new consumer learns the entire tag list from the birth message. No CSV of tag names emailed between integrators.
- **Self-describing types.** Metrics carry real datatypes (Int32, Float, Boolean, String, DateTime) rather than ambiguous bytes.
- **Bandwidth efficiency.** After birth, payloads are protobuf with integer aliases — typically a fraction of equivalent JSON.
- **Guaranteed state.** `NDEATH` is registered as the MQTT will message, so an ungraceful dropout is detected by the broker and announced. Sequence numbers (`seq` , 0-255 rolling) let a consumer detect a missed message and request a full refresh via `NCMD` / `Node Control/Rebirth` .

Sparkplug constraints to plan for

- Requires **QoS 0 for DATA** and **QoS 1 for BIRTH/DEATH/CMD** per spec, and forbids retained messages on Sparkplug topics — state comes from birth certificates instead, which surprises people who expect `retain` to work.
- The `group_id` / `edge_node_id` structure is flatter than a full plant hierarchy; the physical hierarchy usually goes into metric *names* (`Line3/Filler/Motor1/Temperature`).
- Protobuf payloads are not human-readable in a generic MQTT client — you need a Sparkplug-aware tool to debug.

Security posture

MQTT provides **no security of its own**. Everything depends on configuration:

- **Transport**: TLS on port 8883. Without it, credentials and payloads are plaintext on the wire. Verify the broker certificate — a client that skips verification is trivially machine-in-the-middled.
- **Authentication**: username/password in CONNECT (useless without TLS), or X.509 client certificates (much better for fixed devices), or MQTT 5 enhanced authentication with SASL-style challenge/response.
- **Authorisation**: per-topic ACLs on the broker, controlling publish and subscribe separately. This is the control that matters most, and the one most often left wide open. A device should be able to publish only to its own branch of the namespace and subscribe only to its own command topics.

- **No payload integrity**: MQTT does not sign messages. A compromised client with valid credentials can publish any value to any topic its ACL allows.

Default-deny ACLs, per-device credentials (never one shared account), and a broker that is not directly exposed to the internet cover most of the risk.

When to use it

Good for: telemetry from many devices to many consumers; unreliable or metered links (it reconnects and resumes gracefully); deployments where the consumer set changes; plant-to-cloud; Unified Namespace implementations.

Bad for: closed-loop control of any kind; anything needing bounded latency (the broker hop plus TCP retransmission makes worst-case timing unbounded); very large payloads such as video or waveform files — publish a reference to object storage instead; and request/response-heavy workloads, where a REST API or OPC UA is simpler.

Pitfalls

- **Wildcard subscription to `#` on a busy broker.** You will receive everything, including other sites, and probably fall behind. Subscribe narrowly.
- **One client ID used by two devices.** The broker enforces uniqueness by disconnecting the older session, so two devices sharing an ID will knock each other offline in an endless loop. Derive client IDs from serial numbers.
- **Assuming QoS 1 means no duplicates.** It means *at least* once. Deduplicate or be idempotent.
- **Retained messages accumulating forever.** Every topic ever published with retain keeps a message until explicitly cleared. Decommissioned devices leave ghosts in the namespace; clear them with a zero-length retained publish.
- **Keepalive longer than a NAT or firewall idle timeout.** The device thinks it is connected, the broker thinks so too, and the middlebox has silently dropped the flow. Keep keepalive below the shortest idle timeout on the path (often 300 s).
- **No backpressure plan at the edge.** When the uplink fails, where do readings go? Size a local store-and-forward buffer deliberately and decide what to discard when it fills.
- **Treating the broker as stateless.** With persistent sessions and retained messages it holds real state that needs backup and capacity planning.

03 — OPC UA

What it is and where it sits

OPC UA (IEC 62541) is the standard way to expose industrial data **with its meaning attached**. Where Modbus gives you register 40001 and MQTT gives you bytes, OPC UA gives you a browsable, typed, self-describing address space: a `MotorTemperature` variable of type `Double`, in engineering units of degrees Celsius, with a timestamp, a quality status, and a documented relationship to the motor object it belongs to.

It is platform-independent (unlike OPC Classic, which was built on Microsoft COM/DCOM and was notoriously painful to secure and tunnel across subnets) and spans Level 1 to Level 4.

Two distinct communication patterns live under the same name, and conflating them causes most OPC UA confusion:

- **Client/server** — sessions, browsing, reads, writes, method calls, subscriptions. Connection-oriented, TCP, one-to-one.
- **PubSub** — connectionless publish/subscribe over UDP multicast, MQTT or AMQP. One-to-many, added in Part 14.

The information model

Nodes and the address space

Everything in an OPC UA server is a **node** in a graph. Each node has a NodeId, a BrowseName, a NodeClass and typed attributes. Nodes are connected by **references**, which are themselves typed.

Principal node classes:

NodeClass	Role
`Object`	A container representing a thing — a pump, a machine, a folder
`Variable`	Holds a value; this is what you read and write
`Method`	A callable function with input and output arguments
`ObjectType` / `VariableType`	Type definitions that instances point to
`ReferenceType`	The kinds of relationship (`HasComponent` , `Organizes` , …)
`DataType`	Structure and scalar type definitions

Because references are typed, a client can ask structural questions: *what components does this pump have*, *what type is this instance*, *which objects does this alarm refer to*. This is the real advantage over a flat tag list.

NodeIds

A NodeId is a namespace index plus an identifier:

```
ns=2;i=1234          numeric identifier in namespace 2
ns=2;s=Line3.Motor1  string identifier
ns=3;g=<guid>        GUID
ns=0;i=2258          standard node: Server_ServerStatus_CurrentTime
```

Namespace 0 is reserved for the OPC Foundation's standard nodes. **The namespace index is not stable across server restarts or firmware updates** — it is an index into the server's namespace array. Always resolve by namespace *URI* at startup and map to the current index, or your client will break after a controller upgrade. This is one of the most common production bugs in OPC UA clients.

Companion specifications

The base standard defines the mechanics; **companion specifications** define industry-specific models so that two vendors describe the same machine the same way. Widely used ones include PackML and Weihenstephan for packaging, Euromap for plastics machinery, UMATI for machine tools, PLCopen for IEC 61131 program structures, and VDMA models for many machine categories. If one exists for your domain, use it — it is the difference between integration and re-integration.

Services

OPC UA services are grouped into sets. The ones you will actually use:

Service set	Key services	Purpose
Discovery	`FindServers`, `GetEndpoints`	Locate servers and their security options
SecureChannel / Session	`OpenSecureChannel`, `CreateSession`, `ActivateSession`	Establish an authenticated, encrypted channel
View	`Browse`, `BrowseNext`, `TranslateBrowsePathToNodeIds`	Navigate the address space
Attribute	`Read`, `Write`, `HistoryRead`, `HistoryUpdate`	Access values and historical data
Method	`Call`	Invoke a method on an object
MonitoredItem	`CreateMonitoredItems`, `ModifyMonitoredItems`	Register items for change notification
Subscription	`CreateSubscription`, `Publish`	Receive batched notifications

Subscriptions: the part that is usually misconfigured

Do not poll an OPC UA server with repeated `Read` calls. Create a **subscription** and let the server notify you. Three parameters govern behaviour:

- **Publishing interval** (subscription level) — how often the server sends a batch of notifications.
- **Sampling interval** (per monitored item) — how often the server checks the value internally. May be faster than the publishing interval; samples are queued.
- **Queue size and discard policy** (per monitored item) — how many samples are held between publishes, and whether the oldest or newest is dropped on overflow.

A worked example: sampling at 100 ms, publishing at 1 s, queue size 10 gives you all ten samples per second delivered in one packet — full resolution at one tenth the network overhead. Queue size 1 under the same settings gives you only the latest value and silently discards nine out of ten samples. Choosing queue size 1 while expecting complete data is a frequent and hard-to-spot error.

Also set a **deadband** (absolute or percentage) on noisy analogue values, or a fluctuating last digit will generate notifications continuously.

Data value: what a read actually returns

Every value comes with more than a number:

- **Value** — typed, possibly a structure or array
- **StatusCode** — `Good`, `Uncertain`, `Bad`, with specific sub-codes such as `Bad_SensorFailure` or `Uncertain_LastUsableValue`
- **SourceTimestamp** — when the source produced it
- **ServerTimestamp** — when the server processed it

Propagate the StatusCode. A gateway that maps OPC UA to MQTT and throws away status converts a known sensor failure into a plausible-looking reading, which is worse than no reading.

OPC UA PubSub (Part 14)

Client/server is excellent for engineering and modelling but has a per-session cost and a request/response latency floor. PubSub addresses one-to-many and faster exchange:

- **Publisher** sends `NetworkMessage`s containing `DataSetMessage`s, described by `DataSetMetaData`.
- **Transport** is UDP multicast (`opc.udp://`) for low latency on a local network, or MQTT/AMQP as a broker-mediated transport for routed and cloud-bound traffic.
- **No session**, so publishers do not care how many subscribers exist.
- Combined with TSN, this is the basis of deterministic OPC UA for controller-to- controller traffic.

Choose PubSub for many-to-many cyclic data and controller-to-controller exchange; choose client/server when you need browsing, methods, history or write confirmation.

Security posture

OPC UA has the most thorough security model of any protocol in this collection, and it is genuinely good when actually switched on.

Layers

1. **Transport** — `opc.tcp` with the UA Secure Conversation protocol, or HTTPS.
2. **Message security mode** — `None`, `Sign`, or `SignAndEncrypt`. Signing gives integrity and authenticity; encryption adds confidentiality.
3. **Application authentication** — server and client each hold an X.509 certificate and must place the other's in a trust list. Mutual authentication is the default design, not an add-on.
4. **User authentication** — anonymous, username/password, X.509 user certificate, or an issued token (JWT, Kerberos).
5. **Authorisation** — per-node read/write/browse permissions and role-based access control (RBAC) in newer versions.

Security policies

Policy names encode the cryptographic suite, for example `Basic256Sha256`, `Aes128_Sha256_RsaOaep` and `Aes256_Sha256_RsaPss`. The older `Basic128Rsa15` and `Basic256` are **deprecated and should be disabled** — they rely on SHA-1 and PKCS#1 v1.5. Prefer `Aes256_Sha256_RsaPss`, falling back to `Basic256Sha256` for older equipment.

What goes wrong in practice

- `SecurityPolicy#None` **left enabled** so commissioning engineers can connect with any tool. It is almost always still enabled years later. Disable the `None` endpoint in production.
- **Certificate trust by blind acceptance.** Clients that auto-trust any presented certificate discard the entire authentication model. Provision trust lists deliberately.
- **Certificate expiry.** UA certificates expire, often after 1-5 years, and an expired one stops the connection dead — typically at 02:00 on a Sunday. Track expiry dates and plan rotation; a GDS (Global Discovery Server) automates this at scale.
- **Anonymous user tokens with full write access.** Separate application authentication from user authorisation; a trusted client is not the same as a privileged user.

When to use it

Good for: exposing a machine or line in a way others can discover and interpret; vendor-neutral integration between PLC/DCS and MES or historian; replacing OPC Classic; anywhere you want a data model rather than a tag list; secure access across network zones.

Bad for: very constrained devices (a full UA stack is heavy, though UA Nano/Micro profiles and `opc.ua.binary` over constrained transports help); hard real-time motion control with client/server; high-cardinality cloud fan-out, where MQTT/Sparkplug is usually more economical — the common pattern is OPC UA inside the plant, MQTT northbound.

Pitfalls

- **Hard-coding namespace indices.** Resolve by namespace URI every time you connect.
- **Browsing the whole address space on every start.** Large servers hold hundreds of thousands of nodes; cache the browse result and refresh on a model-change event.
- **Ignoring `BadTooManySessions` / `BadTooManyMonitoredItems`.** Embedded servers in PLCs often cap sessions at 5-10 and monitored items in the low thousands. Share one client connection across consumers rather than letting every tool open its own.
- **Reading each tag with a separate `Read` call.** Batch node IDs into one request; the per-call overhead dominates otherwise.
- **Expecting consistent performance from embedded servers.** An OPC UA server running on the PLC CPU competes with the control task. Aggressive subscriptions can extend the scan time of the machine itself.
- **Forgetting that `Write` may be refused silently by a gateway.** Always check the returned StatusCode rather than assuming success.

04 — Modbus

What it is and where it sits

Modbus is the oldest protocol here (Modicon, 1979) and still the most widely implemented. It is a simple request/response protocol in which a client (historically "master") reads and writes numbered registers in a server (historically "slave").

It has no data model, no discovery, no security and no timestamps. It survives because it is trivial to implement, royalty-free, and supported by essentially every piece of industrial equipment ever made. For reading a flow meter or a power monitor, it is often the right answer precisely because it is boring.

Three variants:

Variant	Physical layer	Framing	Addressing
Modbus RTU	RS-485 / RS-232	Binary, CRC-16, silence-based delimiting	1-247 unit ID on a shared bus
Modbus ASCII	RS-485 / RS-232	Hex text, LRC, `:` start and CRLF end	1-247 unit ID
Modbus TCP	Ethernet	MBAP header, no checksum (TCP covers it)	IP address + unit ID

The four data tables

Modbus exposes four separate address spaces. They are genuinely separate — coil 1 and holding register 1 are unrelated storage.

Table	Type	Access	Function codes	Classic numbering
Discrete Inputs	1 bit	Read-only	2	10001-19999
Coils	1 bit	Read/write	1, 5, 15	00001-09999
Input Registers	16 bit	Read-only	4	30001-39999
Holding Registers	16 bit	Read/write	3, 6, 16	40001-49999

The off-by-one that bites everyone

Documentation written for humans uses 1-based "classic" numbering; the protocol on the wire uses 0-based addresses. Holding register **40001** is transmitted as address **0x0000**.

So when a vendor manual says "power factor is in register 40107", the request you must send contains address 106. Worse, some vendors document the wire address directly, and some document `4x107` meaning "holding register 107" which is wire address 106.

Always confirm with a known value. Read the device's firmware revision or serial number register first — if it comes back as the documented value, your offset convention is right. Do not debug a scaling problem before confirming the offset.

Common function codes

Code	Name	Notes
1	Read Coils	Up to 2000 bits
2	Read Discrete Inputs	Up to 2000 bits
3	Read Holding Registers	Up to 125 registers per request
4	Read Input Registers	Up to 125 registers per request
5	Write Single Coil	Value `0xFF00` = on, `0x0000` = off
6	Write Single Register	
15	Write Multiple Coils	
16	Write Multiple Registers	Up to 123 registers
23	Read/Write Multiple Registers	One round trip for both

An error response sets the high bit of the function code (`0x83` for a failed FC 3) and returns an exception code:

Exception	Meaning	Usual real cause
1	Illegal function	Device does not support that FC
2	Illegal data address	Register does not exist — usually your off-by-one
3	Illegal data value	Value outside the device's accepted range
4	Server device failure	Internal fault in the device
6	Server device busy	Retry later; common during device config writes

Data beyond 16 bits

Modbus registers are 16 bits. Anything larger is spread over consecutive registers, and **the specification does not define the order**. This is the single biggest source of Modbus integration pain.

A 32-bit float 123.456 (IEEE 754 `0x42F6E979`) may appear as:

Convention	Register N	Register N+1	Name
Big-endian	`0x42F6`	`0xE979`	ABCD
Little-endian	`0x79E9`	`0xF642`	DCBA
Big-endian byte swap	`0xF642`	`0x79E9`	BADC
Little-endian byte swap	`0xE979`	`0x42F6`	CDAB (word-swapped)

CDAB — correct byte order within each register, registers swapped — is extremely common because many devices are internally little-endian at the 32-bit level.

How to resolve it quickly: read a register whose value you know and can recognise. A temperature reading of roughly 25 that decodes as `1.3e-19` or `4.6e18` is a byte order problem, not a scaling problem. Most Modbus clients offer a word-order toggle; flip it before touching anything else.

Similar ambiguity applies to 32-bit integers, 64-bit values and to strings packed two characters per register.

Scaling

Registers carry raw integers. A temperature of 124.7 degC is typically transmitted as `1247` with a documented divisor of 10. Scaling factors live only in the vendor manual and must be hand-configured in every client. There is no way to discover them, and no units travel with the value. This is exactly what OPC UA and Sparkplug exist to fix.

Configuration that matters

Modbus RTU on RS-485

- **Baud, parity, stop bits, data bits** must match on every node. 19200 8N1 and 9600 8E1 are both common defaults; a mismatch produces garbage or silence.
- **Unit IDs must be unique** on the bus. Two devices answering as ID 1 produce corrupted frames that look like noise.
- **Termination**: 120 ohm resistors at both physical ends of the bus only. Over- terminating and under-terminating both cause reflections and intermittent CRC errors — the classic "works at 9600, fails at 115200" symptom.
- **Bias resistors** hold the idle line in a defined state; without them some transceivers see false start bits.
- **Topology**: daisy chain. Star and stub topologies work until they do not, typically on a hot afternoon.
- **Silent interval**: frames are delimited by 3.5 character times of silence, so an interval that is too short merges frames.
- **Turnaround delay**: half-duplex RS-485 needs the client to stop driving the line before the server replies. Devices with slow transceivers need an added delay.

Modbus TCP

- **Port 502** by default.
- **Unit ID** is still present in the MBAP header. For a native Ethernet device it is usually ignored or must be 1 or 255; for a **gateway** bridging to serial, it selects the downstream RTU device. Getting this wrong is a common gateway failure.
- **One request at a time per connection** for most devices. Pipelining several outstanding transactions, though allowed by the Transaction Identifier field, breaks many embedded implementations.
- **Connection limits** are low — often 1 to 4 concurrent sockets. A polling tool left running on an engineer's laptop can lock out the SCADA system.
- **No CRC** at Modbus level; TCP checksums are relied on.

Security posture

Modbus has **no security whatsoever**. No authentication, no authorisation, no integrity, no encryption. Any host that can reach port 502 can write any register, including ones that start a motor or change a setpoint. Function code 8 (diagnostics) can in some implementations force a listen-only mode or restart communications.

Modbus/TCP Security (a later addendum adding TLS on port 802 with X.509) exists but is rarely implemented in the field.

Therefore protection must be external:

- Put Modbus devices on an isolated VLAN or segment with no route from general IT.
- Use a firewall that permits only the specific client IPs to reach port 502.
- Prefer a **data diode or read-only gateway** where only monitoring is needed — this removes the write risk structurally rather than by policy.
- Use a protocol-aware firewall if available, to allow read function codes and block write function codes.

- Never expose port 502 to the internet. Internet-wide scans find these constantly, and they are unauthenticated by design.

When to use it

Good for: reading simple instruments — power meters, flow meters, temperature controllers, VFDs, inverters; small point counts; situations where the device offers nothing else; quick integration where a data model is unnecessary.

Bad for: anything needing semantics, discovery or timestamps; high tag counts (register maps become unmanageable spreadsheets); event-driven data, since Modbus can only poll; writing to safety-relevant outputs over a shared network; multi-consumer access to one device, given the tiny connection limits.

Pitfalls

- **The off-by-one.** Verify against a known register before anything else.
- **Word order on 32-bit values.** Verify against a known value before anything else.
- **Polling too fast.** A meter that integrates over 250 ms will return the same value if polled at 50 ms, and may slow down or stop responding under the load.
- **Requesting a span that crosses an unimplemented gap.** Reading registers 100-120 fails entirely with exception 2 if 110 does not exist. Read contiguous implemented blocks.
- **Exceeding 125 registers in one read.** The limit comes from the 253-byte PDU; many clients do not check it and simply produce a malformed frame.
- **Trusting a stale value.** Modbus has no quality flag and no timestamp. If a device stops updating internally but still answers, you will read the last value forever and never know. Add a heartbeat or counter register to your poll and alarm on it failing to change.
- **Assuming the gateway's unit ID mapping is transparent.** Serial-to-TCP gateways vary in whether they map unit ID, port number or IP alias to the downstream device.

05 — Industrial Ethernet

The core problem

Standard Ethernet with TCP/IP is excellent at throughput and hopeless at determinism. Collisions are gone with switches, but queuing delay, switch buffering, TCP retransmission and OS scheduling all add unbounded jitter. A servo drive that needs a new position setpoint every 250 microseconds, with less than a microsecond of variation, cannot use it as-is.

Each industrial Ethernet protocol solves this differently, and the approach determines what hardware you need and what the network can do:

Approach	How determinism is achieved	Examples
Prioritised frames on standard switches	VLAN priority, small cyclic frames, bypass TCP/IP	PROFINET RT, EtherNet/IP
Time-scheduled transmission	Frames sent in reserved time slots, synchronised clocks	PROFINET IRT, Sercos III, TSN
Summation frame / on-the-fly processing	One frame passes all nodes; each reads and writes in transit	EtherCAT
Time-slotted master control	Master grants each node an exclusive send window	POWERLINK

PROFINET

PROFINET (PROFIBUS & PROFINET International) is dominant in Siemens-centric plants and across much of Europe.

Conformance and real-time classes

Class	Mechanism	Cycle time	Hardware needed
NRT	TCP/UDP/IP	> 100 ms	Any Ethernet
RT (Class A/B)	EtherType `0x8892`, VLAN priority, no IP stack	1-10 ms	Standard managed switch
IRT (Class C)	Scheduled transmission, synchronised clocks, reserved bandwidth	31.25 us - 1 ms	PROFINET-aware switch ASIC

Conformance Classes A, B, C and D describe certified capability levels; CC-C is required for IRT, and CC-D covers TSN-based PROFINET.

How it works

- **Device description** is a **GSDML** file (XML). It declares modules, submodules, I/O data sizes and parameters. The engineering tool uses it to build the configuration; without the right GSDML version you cannot commission the device.
- **Addressing** uses a device **name** (not primarily an IP) assigned via **DCP** (Discovery and Configuration Protocol). The controller resolves name to MAC at startup. A replaced device must be given the correct name or the controller will not recognise it — this is the most common PROFINET field task.
- **Cyclic I/O** exchanges a fixed process image each cycle. The controller writes outputs, the device writes inputs; both continue regardless of change.
- **Acyclic services** (parameter read/write, diagnostics, alarms) use UDP/IP on the same wire.
- **PROFIsafe** carries safety data (up to SIL 3) inside the standard channel using a safety CRC and watchdog — a "black channel" approach, so the underlying network need not be safety-certified.

- **Media Redundancy Protocol (MRP)** gives ring topology recovery, typically in tens of milliseconds; **MRPD** achieves bumpless redundancy for IRT.

EtherNet/IP

EtherNet/IP (ODVA) is dominant in North America and in Rockwell/Allen-Bradley installations. The "IP" means **Industrial Protocol**, not Internet Protocol.

- Built on **CIP** (Common Industrial Protocol), the same application layer used by DeviceNet and ControlNet. Devices are modelled as **objects** with classes, instances and attributes.
- **Explicit messaging** uses TCP port 44818 for configuration, diagnostics and non-time-critical reads — request/response, higher overhead.
- **Implicit messaging** uses UDP port 2222 for cyclic I/O, with a configured **Requested Packet Interval (RPI)**. Typical RPI is 1-20 ms.
- **Device description** is an **EDS** file.
- **CIP Safety** provides SIL 3 safety; **CIP Sync** uses IEEE 1588 for time synchronisation; **CIP Motion** adds motion control.
- **CIP Security** adds authentication and integrity to CIP — relatively recent and not yet universally supported.

A frequent planning error is sizing multicast traffic. Implicit messaging has historically used multicast for producer-to-consumer I/O, which floods switches that lack IGMP snooping. Enable IGMP snooping with a querier, or configure unicast connections where the hardware allows.

EtherCAT

EtherCAT (EtherCAT Technology Group, originally Beckhoff) takes a fundamentally different approach and achieves the best cycle times with ordinary cabling.

The summation frame

The master sends **one** Ethernet frame that passes through every slave in the logical ring. Each slave reads its own output data and inserts its input data **as the frame passes through the hardware**, with a delay of nanoseconds. The frame returns to the master containing the complete updated process image.

Consequences:

- Extremely efficient: one frame can serve hundreds of nodes, so bandwidth use is near-optimal and 100 Mbit/s is usually ample.
- Cycle times of 50-100 microseconds for substantial node counts are routine.
- **Distributed Clocks** synchronise slaves to well under 1 microsecond, which is what makes coordinated multi-axis motion possible.
- Slaves need an EtherCAT slave controller ASIC; you cannot make an EtherCAT slave in software on a normal NIC. The master, by contrast, can be pure software on a standard NIC.
- It uses EtherType `0x88A4` with no IP. It is **not routable** and cannot cross a normal L3 boundary. `EoE` (Ethernet over EtherCAT) tunnels normal traffic when needed.
- Topology is physically flexible (line, tree, star with junctions) but always a logical ring. Cable redundancy is available by closing the ring back to a second master port.

Device description is an **ESI** file (XML). **FSoE** (Fail Safe over EtherCAT) carries SIL 3 safety data.

POWERLINK and Sercos III

Ethernet POWERLINK (B&R / EPSG) uses a managing node that polls each controlled node in a strictly scheduled cycle, so no two nodes transmit simultaneously and collisions cannot occur. It is open source on the master side, achieves cycle times down to around 100-200 microseconds, and runs on standard Ethernet hardware.

Sercos III combines a telegram-based real-time channel with an unused-bandwidth channel for IP traffic, reaching 31.25 microsecond cycles. It is strong in machine tools and motion applications.

Both are more specialised than PROFINET/EtherNet/IP/EtherCAT; choose them when the machine builder or drive vendor already standardises on them.

TSN: the convergence story

Time-Sensitive Networking is a set of IEEE 802.1 amendments that push determinism into standard Ethernet, so that control traffic and ordinary IT traffic can share infrastructure safely.

Key pieces:

Standard	Function
802.1AS	Time synchronisation (a profile of IEEE 1588 / gPTP)
802.1Qbv	Time-aware shaper — scheduled transmission gates per queue
802.1Qbu / 802.3br	Frame pre-emption — a long frame yields to an urgent one
802.1Qav	Credit-based shaper for bandwidth reservation
802.1Qcc	Centralised configuration of streams
802.1CB	Frame replication and elimination for reliability

TSN is a *layer-2 toolbox*, not a protocol in itself. PROFINET over TSN, EtherNet/IP over TSN and OPC UA PubSub over TSN all exist. The promise is one physical network carrying motion control, process data and video. The practical caveat is that TSN requires TSN-capable switches throughout the path and careful central scheduling; a single non-TSN switch in the middle voids the guarantees.

Choosing between them

In practice the decision is usually made for you:

- **Your PLC vendor decides.** Siemens leads to PROFINET; Rockwell to EtherNet/IP; Beckhoff and most motion-centric machine builders to EtherCAT.
- **Your drives decide.** If the servo drives you need only speak EtherCAT, that settles the machine network.
- **Your customer's plant standard decides.** Many large manufacturers mandate one protocol for maintainability, and that is a legitimate engineering reason.

If genuinely free to choose: EtherCAT for motion and the fastest cycles; PROFINET for large plant-wide installations with mixed vendors in Europe; EtherNet/IP in North America or an existing Rockwell estate.

Security posture

Fieldbus protocols at this level were designed for physically isolated networks and their native security is minimal:

- Cyclic frames are unauthenticated. Anything on the segment that can emit the right EtherType can inject I/O data.
- PROFINET DCP lets any host on the segment rename or re-address devices. A DCP "flash LED" or "reset to factory" command needs no credentials.

- EtherCAT has no access control at all; physical access to the ring is total control.
- Safety layers (PROFIsafe, CIP Safety, FSoE) protect against **random faults and corruption**, not against a deliberate attacker. They are functional safety, not cybersecurity. This distinction matters and is often blurred in vendor material.

Controls that actually work here are architectural: strict segmentation, with the machine network behind a controller or firewall and never bridged to IT; physical security of panels and switch ports; disabling unused switch ports; and restricting engineering access to a hardened, monitored jump host. Add CIP Security or PROFINET security extensions where the hardware supports them, but do not rely on them as a substitute for segmentation.

Pitfalls

- **Using an unmanaged switch for RT traffic.** No VLAN priority handling means no prioritisation; IRT will not work at all.
- **Connecting a laptop to the machine network to "just have a look".** Windows broadcast traffic and discovery chatter have stopped production lines.
- **Mixing protocols on one segment without separation.** PROFINET and EtherNet/IP can coexist electrically, but multicast storms and bandwidth contention will eventually break the slower budget.
- **Replacing a device without setting its device name (PROFINET) or station alias.** The controller will not adopt it.
- **Forgetting that EtherCAT cable order is topology.** Moving a slave to a different port changes its position in the ring and therefore its address.
- **Ignoring IGMP snooping with EtherNet/IP multicast.** Works fine on the bench with three nodes; floods the network at forty.
- **Assuming GSDML/EDS/ESI files are interchangeable between firmware versions.** They are version-matched; a mismatch shows up as cryptic configuration errors.

06 — Fieldbus & Device Level

These are the protocols closest to the physical process. Many predate Ethernet and remain installed in enormous numbers — a refinery or water utility commissioned in 1998 is still running, and its instruments still speak these. Any realistic IIoT project is partly an exercise in extracting data from them.

HART

HART (Highway Addressable Remote Transducer) is the cleverest piece of backward-compatibility engineering in industrial instrumentation.

The installed base of process transmitters uses a **4-20 mA current loop**: the transmitter varies current in proportion to the measurement, and this analogue signal is simple, robust and intrinsically safe. HART superimposes a **frequency-shift-keyed digital signal** (1200 Hz and 2200 Hz, 1200 bit/s) on top of the same two wires. The FSK signal averages to zero current, so the analogue reading is undisturbed.

The result: a 1990s control system keeps reading 4-20 mA while a modern asset management tool simultaneously reads the device's serial number, calibration data, secondary variables and diagnostics over the same pair.

- Roughly 2-3 digital updates per second — not a control-rate protocol.
- Up to four process variables plus device status per device.
- **Multidrop mode** allows up to 15 devices digitally on one pair, but the analogue current is then fixed at 4 mA and unusable — rarely used.
- **HART-IP** carries HART payloads over Ethernet/UDP, which is the usual route for getting HART diagnostics into an IIoT platform.
- Device capability is described by a **DD** (Device Description) or **FDI** package.

Why this matters for IIoT: a vast amount of diagnostic data already exists in installed transmitters and is simply never read, because the DCS only samples the analogue value. A HART multiplexer or a HART-capable I/O card plus HART-IP is often the cheapest possible condition-monitoring project — no new sensors required.

PROFIBUS

Variant	Use	Physical layer
PROFIBUS DP	Factory automation, fast I/O	RS-485, 9.6 kbit/s - 12 Mbit/s
PROFIBUS PA	Process industry instrumentation	MBP, 31.25 kbit/s, bus-powered, intrinsically safe

- Token-passing between masters, polling of slaves; cycle times of a few milliseconds for modest node counts on DP.
- Device description is a **GSD** file.
- **PA** carries power and data on the same pair and supports hazardous areas, which is why it persists in chemical and oil-and-gas plants.
- Baud rate limits cable length strictly: 12 Mbit/s allows only 100 m per segment, while 9.6 kbit/s allows 1200 m. Repeaters extend this but add delay.
- **Termination is active** and requires powered terminators at segment ends; an unpowered terminator is a frequent cause of flaky PROFIBUS segments.

Migration path is PROFINET, usually via a PROFIBUS-to-PROFINET proxy that keeps the existing field wiring.

Foundation Fieldbus

FOUNDATION Fieldbus H1 (31.25 kbit/s, bus-powered, intrinsically safe) competes with PROFIBUS PA in process plants. Its distinguishing idea is **control in the field**: function blocks (PID, AI, AO) execute inside the instruments themselves, and a loop can keep running even if the host controller is lost. A scheduled, publisher/subscriber link layer provides deterministic block execution.

This is powerful but makes the configuration genuinely complex, and the commissioning skill set is specialised. New installations increasingly choose PROFIBUS PA, HART or Ethernet-APL instead.

Ethernet-APL

Ethernet-APL (Advanced Physical Layer) is the significant recent development for process industries: 10 Mbit/s Ethernet over a single twisted pair, delivering power to the device, with cable runs to 1000 m and intrinsic safety for Zone 0 / Division 1 hazardous areas.

It lets process instruments speak Ethernet protocols — typically **PROFINET** or **OPC UA/HART-IP** — directly, with the same two-wire installation practice and hazardous-area approvals the industry already uses. For greenfield process plants it removes the historical reason for separate fieldbus physical layers, and it is the main reason to expect OPC UA to reach down to individual transmitters over the next decade.

CAN-based protocols

CAN (Controller Area Network) is a robust multi-master serial bus with collision resolution by message priority and strong error detection. CAN itself defines only layers 1-2; higher-layer protocols add meaning.

CANopen

- Device profiles standardise how classes of device (drives, I/O modules, encoders) expose their data.
- The **Object Dictionary** indexes all parameters (16-bit index, 8-bit subindex).
- **PDOs** (Process Data Objects) carry real-time data, up to 8 bytes, unconfirmed.
- **SDOs** (Service Data Objects) provide confirmed access to any dictionary entry for configuration.
- **NMT** (Network Management) handles state (pre-operational, operational, stopped) and heartbeats.
- Common in machinery, medical devices, off-highway vehicles and maritime.

SAE J1939

- CAN-based standard for heavy-duty vehicles: trucks, buses, agricultural and construction machines, and many generator sets.
- Data is organised into **PGNs** (Parameter Group Numbers) containing **SPNs** (Suspect Parameter Numbers), with standardised definitions for engine speed, coolant temperature, fuel rate, diagnostic trouble codes and so on.
- 250 kbit/s classically, 500 kbit/s increasingly.
- For IIoT this is the protocol behind most telematics: a J1939 reader on a generator or a fleet vehicle yields standardised engine data without vendor negotiation.

IO-Link

IO-Link (IEC 61131-9) is point-to-point digital communication over the existing three-wire sensor cable, between a sensor/actuator and an **IO-Link master**. The master then connects upward via PROFINET, EtherNet/IP or EtherCAT.

Why it is genuinely useful:

- A simple proximity or photoelectric sensor gains **parameters, diagnostics and identity** — temperature, operating hours, signal quality, manufacturer and serial number — over the same cheap unshielded cable.
- **Automatic parameter download**: replace a failed sensor and the master writes its configuration automatically. No hand-setting of potentiometers or teach buttons.
- Device description is an **IODD** file.
- Cable lengths up to 20 m, unshielded, standard M12 connectors.

IO-Link is one of the highest-value, lowest-friction additions to an existing machine for condition monitoring, because it reuses wiring already in place.

AS-Interface

AS-i (Actuator Sensor Interface) is a two-wire bus carrying both power and data for simple binary devices, with a distinctive yellow flat cable pierced by self-sealing connectors. Up to 62 slaves, cycle time around 5-10 ms. It is cheap and quick to install for large numbers of simple on/off devices, and typically appears behind a gateway to PROFINET or EtherNet/IP.

BACnet and building systems

BACnet (ASHRAE 135, also ISO 16484-5) is the dominant building automation protocol: HVAC, lighting, access control, metering.

- Object-oriented model: Analog Input, Binary Output, Schedule, Trend Log and similar standard object types with defined properties.
- **BACnet/IP** (UDP port 47808) is now most common; BACnet MS/TP over RS-485 is widely installed for field devices.
- Supports change-of-value subscriptions, scheduling, trending and alarming natively.

It matters for IIoT because facility energy data — chillers, air handling, submetering — usually lives in BACnet while production data lives in PROFINET or Modbus. Any serious plant energy-efficiency project has to bridge both.

DNP3 and utility protocols

DNP3 (IEEE 1815) dominates electric utilities, water and wastewater SCADA in North America, with **IEC 60870-5-101/104** filling the same role in Europe.

Features that distinguish it from Modbus:

- **Timestamped events with sequence-of-events buffering.** A device records *what changed and when*, buffers it, and reports on request — so a brief outage does not lose the event history. This is essential for fault analysis on a power network.
- **Unsolicited responses**: the outstation can report spontaneously rather than waiting to be polled.
- **Data quality flags** accompany values.
- **DNP3 Secure Authentication (SAv5)** adds challenge-response authentication — one of the few retrofitted security layers in this group that is actually deployed.

IEC 61850 is the substation automation standard, worth knowing for its architecture: a rich object model with standardised logical nodes, plus **GOOSE** messages (multicast layer-2, for protection trips in under 4 ms) and **sampled values** for digitised current and voltage waveforms.

Getting data out of legacy buses

The realistic options, roughly in order of preference:

1. **Read it from the controller.** The PLC or DCS already polls the fieldbus. Exposing an OPC UA server or Modbus TCP slave on the controller is usually the least invasive route, and preserves the engineering that is already correct.
2. **Protocol gateway.** A dedicated device bridging PROFIBUS, CANopen, HART or AS-i to Modbus TCP, OPC UA or MQTT. Cheap and effective; adds a translation hop and a device to maintain.
3. **Tap the bus directly.** A passive listener on RS-485 or CAN can read traffic without disturbing it. Attractive because it cannot write, but you must reverse the address and scaling conventions yourself.
4. **Add parallel instrumentation.** Where the existing system cannot be touched — validated pharmaceutical or safety-related installations — fitting independent sensors with their own IIoT path is sometimes the only permissible option.

Whichever route, do not disturb the control network to gather monitoring data. Adding a polling client to a loaded PROFIBUS segment can extend its cycle time enough to affect control.

Pitfalls

- **Adding a polling master to a live fieldbus.** Token and cycle budgets are finite; monitoring traffic can degrade control.
- **Assuming HART data is available.** It requires HART-capable I/O cards or a multiplexer. Many installations have HART instruments wired to plain analogue inputs, so the digital channel exists but nothing is listening.
- **Ignoring intrinsic safety requirements.** In hazardous areas, energy limits are certified. Connecting a non-certified gateway into an IS circuit invalidates the area classification — a safety and legal issue, not a technical preference.
- **Mismatched GSD/EDS/IODD versions** after a device replacement.
- **Unpowered or missing bus terminators** on PROFIBUS and RS-485 segments.
- **Treating J1939 or CANopen addresses as fixed.** Node IDs and PGN availability vary by engine or device manufacturer and model year, despite the standard.

07 — CoAP, AMQP & DDS

Three protocols that solve problems MQTT does not: REST semantics on tiny battery-powered devices (CoAP), reliable transactional enterprise messaging (AMQP), and real-time data-centric distribution with QoS contracts (DDS).

CoAP

Constrained Application Protocol (RFC 7252) is REST for devices too small for HTTP and TCP. It keeps the familiar model — GET, POST, PUT, DELETE on URIs, response codes, content types — but runs over **UDP** with a 4-byte fixed header.

Why not just use HTTP?

An HTTP request with headers is typically several hundred bytes; TCP needs a three-way handshake and keeps connection state; TLS adds a multi-round-trip handshake. On a device with 32 kB of RAM running from a coin cell over a lossy radio, all of that is prohibitive. A CoAP exchange can be a single UDP datagram in each direction, with the radio powered for a few milliseconds.

How it works

- **Message types**: Confirmable (CON, requires ACK), Non-confirmable (NON, fire and forget), Acknowledgement (ACK), Reset (RST). Reliability is implemented at the CoAP layer with exponential backoff, rather than relying on TCP.
- **Response codes** mirror HTTP: `2.05 Content`, `4.04 Not Found`, `5.03 Service Unavailable`.
- **Observe** (RFC 7641) is the important extension: a client registers interest in a resource and the server pushes updates as it changes. This gives publish/subscribe behaviour without a broker.
- **Block-wise transfer** (RFC 7959) moves payloads larger than one datagram in numbered blocks — necessary for firmware updates.
- **Resource discovery** via `GET /.well-known/core` returning CoRE Link Format, so a client can enumerate what a device offers.
- **Proxying** to HTTP is straightforward because the semantics align, which makes CoAP devices easy to expose to web tooling.
- **CoAP over TCP/TLS/WebSockets** (RFC 8323) exists for traversing networks that block UDP.
- Default ports: **5683** (plaintext), **5684** (DTLS).

Payloads

CoAP carries any content type; in practice **CBOR** (RFC 8949, a compact binary JSON-equivalent) and **SenML** (RFC 8428, a standard sensor measurement format) are usual. SenML is worth knowing because it gives a conventional way to express value, unit, time and name — the semantics MQTT lacks.

Security

DTLS (TLS over UDP) with pre-shared keys, raw public keys or certificates. PSK is common on constrained devices since certificate handling is heavy. **OSCORE** (RFC 8613) is an alternative providing object-level security that survives proxying, which DTLS cannot.

When to use it

Good for: battery-powered sensors; 6LoWPAN/Thread networks; large fleets of constrained devices; situations where you want REST semantics end to end; **LwM2M** device management, which is built on CoAP and is the standard way to manage cellular IoT fleets.

Bad for: anything behind NAT without a persistent mapping (UDP bindings expire); plant-floor control; high-throughput data; environments whose firewalls block UDP.

AMQP

Advanced Message Queuing Protocol. Two incompatible things share the name, and confusing them wastes a lot of time:

- **AMQP 0-9-1** — the protocol RabbitMQ is built around. Defines exchanges, queues and bindings as broker-side concepts.
- **AMQP 1.0** — the OASIS and ISO/IEC 19464 standard. Defines only the wire protocol between nodes; broker topology is out of scope. This is what Azure Service Bus, Azure IoT Hub, ActiveMQ and Qpid implement.

What it offers over MQTT

Capability	AMQP 1.0	MQTT
Message metadata	Rich typed headers and annotations	Topic + small set of properties (MQTT 5)
Payload typing	Self-describing type system	Opaque bytes
Transactions	Yes — atomic multi-message commit	No
Flow control	Credit-based, per link	`Receive Maximum` (MQTT 5 only)
Queue semantics	Explicit queues, competing consumers	Shared subscriptions (MQTT 5 only)
Protocol overhead	Higher	Minimal
Footprint	Larger client stacks	Tiny

The decisive features are **transactions** and **credit-based flow control**. If a message represents a financial or inventory event that must not be lost or duplicated, AMQP's transactional semantics are worth the overhead. Credit-based flow control means a consumer explicitly grants the sender permission for N more messages, which makes backpressure precise rather than best-effort.

Model

- **Connection** carries one or more **sessions**; each session carries **links**.
- A link is unidirectional: a **sender** attaches to a target, a **receiver** to a source.
- **Settlement** defines delivery guarantees: at-most-once, at-least-once, or exactly-once via a two-phase disposition exchange.
- Ports: **5672** plaintext, **5671** TLS. Authentication via **SASL** (PLAIN, EXTERNAL for client certificates, and others).

When to use it

Good for: Level 3-4 integration — MES to ERP, order and batch records, anything transactional; cloud ingestion where the platform speaks AMQP natively (Azure IoT Hub uses it as a primary protocol); workloads needing competing consumers over a work queue.

Bad for: constrained devices; very large device fleets, where MQTT's smaller footprint and simpler model win; anything where you only need telemetry fan-out.

DDS

Data Distribution Service (OMG) is different in kind from the others. It is **data-centric** rather than message-centric: participants publish and subscribe to strongly typed **topics** forming a shared **global data space**, and the middleware maintains that state with contractual QoS guarantees. There is no broker.

Core concepts

- **Domain** — an isolation boundary identified by an integer. Participants in different domains never see each other.
- **Topic** — a name plus a data type defined in **IDL**.
- **DataWriter / DataReader** — the typed endpoints.
- **Keys** — a field marked as key makes a topic behave like a table of instances: one `SensorReading` topic keyed on `sensor_id` maintains an independent value and lifecycle per sensor.
- **RTPS** — the Real-Time Publish-Subscribe wire protocol, normally over UDP multicast, which makes DDS implementations interoperable across vendors.
- **Automatic discovery** — SPDP finds participants, SEDP matches readers to writers by topic, type and QoS compatibility. No configuration of addresses.

QoS policies: the real distinguishing feature

QoS is a **contract**. A reader and writer connect only if their requested and offered policies are compatible, so incompatibility surfaces at startup rather than as mysterious data loss later.

Policy	Effect
`RELIABILITY`	`BEST_EFFORT` or `RELIABLE` (with retransmission)
`DURABILITY`	`VOLATILE`, `TRANSIENT_LOCAL`, `TRANSIENT`, `PERSISTENT` — late joiners can receive historical data
`HISTORY`	`KEEP_LAST(n)` or `KEEP_ALL`
`DEADLINE`	Maximum expected interval between samples; violation raises a status
`LIVELINESS`	How writer aliveness is asserted and how quickly loss is detected
`OWNERSHIP`	`EXCLUSIVE` with strength values lets a standby publisher take over automatically
`LATENCY_BUDGET`	Permits batching within a stated latency allowance
`TIME_BASED_FILTER`	Reader-side rate limiting
`PARTITION`	Logical sub-channels within a topic
`RESOURCE_LIMITS`	Bounded memory — essential for embedded determinism

`OWNERSHIP` with `EXCLUSIVE` deserves emphasis: it provides redundant publisher failover as a configuration setting rather than application code, which is why DDS appears in systems that must not stop.

Where it is used

Defence and aerospace (it originated there), air traffic control, naval systems, medical devices, autonomous vehicles, and **ROS 2**, whose entire middleware layer is DDS. If you work on robotics, you are using DDS whether or not you think about it.

DDS Security (a separate OMG specification) adds authentication, access control, cryptography and logging as pluggable components, with permissions expressed in signed governance and permissions documents.

When to use it

Good for: many-to-many high-rate data inside a system; latency in the sub-millisecond to low-millisecond range; applications needing redundancy and failover; robotics and autonomous platforms; cases where QoS must be explicit and enforced.

Bad for: crossing the internet or NAT (multicast discovery does not traverse it; routing services exist but add complexity); constrained 8-bit devices; simple telemetry, where it is substantial overkill; teams without the appetite for its genuine conceptual complexity.

Choosing between the three

Need	Choose
Battery sensor, lossy radio, REST semantics	**CoAP**
Device management for a cellular fleet	**CoAP** (via LwM2M)
Transactional business events, exactly-once	**AMQP 1.0**
Work queue with competing consumers	**AMQP 1.0**
Real-time data sharing inside a robot or vehicle	**DDS**
Publisher redundancy with automatic failover	**DDS**
Plain telemetry from many devices to many consumers	**MQTT** (see 02)

Pitfalls

- **CoAP behind NAT.** UDP mappings expire in as little as 30 seconds on some cellular networks. Either keep traffic device-initiated with short intervals, or use CoAP over TCP.
- **CoAP without congestion awareness.** UDP has no built-in congestion control; a fleet retrying simultaneously after an outage can saturate the uplink. Randomise backoff.
- **Confusing AMQP 0-9-1 with 1.0.** A RabbitMQ client library will not talk to an AMQP 1.0 endpoint without the appropriate plugin or a different library.
- **DDS multicast on a network that does not forward it.** Discovery silently fails and nothing connects. Check IGMP and, where needed, configure a discovery peer list explicitly.
- **DDS QoS mismatch.** A `BEST_EFFORT` writer will not match a `RELIABLE` reader. The reader simply receives nothing, with no error unless you check the matched-publication status. Always log discovery and match events during commissioning.
- **Unbounded DDS history on an embedded target.** `KEEP_ALL` with no `RESOURCE_LIMITS` will eventually exhaust memory. Set limits deliberately.

08 — Wireless & LPWAN

The trade-off that governs every choice

You cannot have long range, high data rate and low power consumption at once. Every wireless technology picks two and sacrifices the third. Decide which two you need before comparing products, because this constraint is physics, not marketing.

Technology	Range	Data rate	Power	Topology	Spectrum
WirelessHART	10-200 m/hop, mesh	250 kbit/s	Years on battery	Mesh	2.4 GHz ISM
ISA100.11a	10-200 m/hop, mesh	250 kbit/s	Years	Mesh	2.4 GHz ISM
Zigbee	10-100 m/hop, mesh	250 kbit/s	Years	Mesh	2.4 GHz, 868/915 MHz
Thread	10-100 m/hop, mesh	250 kbit/s	Years	Mesh, IPv6	2.4 GHz
BLE	10-100 m	125 kbit/s - 2 Mbit/s	Years	Star, mesh	2.4 GHz
Wi-Fi (802.11)	30-100 m	10 Mbit/s - 1 Gbit/s+	Mains or short battery life	Star	2.4/5/6 GHz
LoRaWAN	2-15 km	0.3-50 kbit/s	5-10 years	Star of stars	Sub-GHz ISM
NB-IoT	Cellular coverage	20-250 kbit/s	5-10 years	Star (cellular)	Licensed
LTE-M	Cellular coverage	300 kbit/s - 1 Mbit/s	3-8 years	Star (cellular)	Licensed
5G URLLC	Cellular / private	High	Mains	Star	Licensed / CBRS

Industrial process wireless

WirelessHART (IEC 62591)

The established choice for process instrumentation, and the one to reach for when retrofitting measurement points in a refinery, chemical plant or tank farm.

- **Self-organising, self-healing mesh.** Every mains- or battery-powered field device can route for its neighbours. Add a device and the mesh incorporates it; lose one and traffic reroutes.
- **TDMA with 10 ms time slots and channel hopping** across 15 channels. This is the key reliability mechanism: a narrowband interferer (or a Wi-Fi access point) blocks one channel, and the hopping sequence simply avoids it. Field reliability above 99.9 % is routinely achieved in environments full of metal and RF noise.
- **Blacklisting** permanently excludes channels that conflict with plant Wi-Fi.
- Carries the **same HART command set** as wired HART, so existing asset management tools and device descriptions work unchanged — this is the real reason it won in process industries.
- Security is mandatory and not optional: AES-128 at both network and session layers, with a join key per device and rotating session keys.
- Update rates from 1 s to an hour, configured per device; battery life is driven almost entirely by this choice.

ISA100.11a (IEC 62734)

Technically similar — 2.4 GHz, TDMA, channel hopping, mesh — but built on **6LoWPAN and IPv6**, making it natively IP-routable, and more flexible in its object model and slot timing. It supports tunnelling other protocols, including HART and Modbus.

WirelessHART has the larger installed base in process plants; ISA100 appeals where IPv6 integration or multi-protocol tunnelling matters. Both are credible; the decision is usually driven by which instrument vendors the site already uses.

Short-range mesh and personal-area

Zigbee

IEEE 802.15.4 based mesh with application profiles. Strong in building automation, lighting control and smart metering. Coordinator, router and end-device roles; end devices sleep and parent routers buffer for them. Security is AES-128 with a network key and optional link keys.

Industrially it is less common than WirelessHART because it lacks the mandated channel hopping and the HART compatibility, but it is inexpensive and well supported.

Thread

Also 802.15.4, but a **native IPv6 mesh** (6LoWPAN) with no application layer of its own — Matter and CoAP run on top. Border routers connect it to normal IP networks. Its appeal is that every node is addressable like any other IP host, so standard tooling applies. Mostly a building and consumer technology so far, but architecturally the cleanest of the 802.15.4 family.

Bluetooth Low Energy

- **Connection-oriented** links for configuration and data exchange, plus **advertising** for connectionless broadcast, which is how most asset tags work.
- **GATT** defines services and characteristics — the data model.
- BLE 5.x adds **Coded PHY** (roughly four times the range at reduced rate), **2M PHY** (double rate), and **LE Audio**.
- **Direction finding** (AoA/AoD) in BLE 5.1 enables sub-metre indoor positioning, which is the basis of most real-time location systems for tools, trolleys and people.

Industrial uses: commissioning and configuration via phone or tablet, condition monitoring sensors on motors and pumps, asset tracking, and operator access control. Not for control.

Wi-Fi

Ubiquitous and high bandwidth, with real industrial caveats. Roaming between access points takes tens to hundreds of milliseconds unless 802.11r/k/v fast transition is properly configured — fatal for an AGV or crane that must stay connected while moving. Power consumption rules out battery-powered sensors on anything but a long duty cycle. The 2.4 GHz band is congested and will collide with Zigbee and WirelessHART unless channels are planned together.

Wi-Fi 6/6E improves determinism meaningfully through OFDMA and scheduled access (trigger-based uplink), and 6 GHz brings clean spectrum. It is now a reasonable choice for AGVs and handheld terminals, though still not for closed-loop control.

LPWAN

LoRaWAN

- **Chirp spread spectrum** modulation gives an extraordinary link budget: kilometres of range, and good penetration into basements, pits and meter cabinets.
- **Star-of-stars**: devices talk to gateways, gateways forward to a network server, which routes to an application server. Devices are not addressed individually by the radio layer — any gateway that hears an uplink forwards it, and duplicates are removed centrally.
- **Device classes**:
- **Class A** — uplink-initiated, two short downlink windows after each uplink. Lowest power; downlink latency is up to the next uplink.

- **Class B** — scheduled downlink slots synchronised by gateway beacons. Bounded downlink latency at moderate power cost.
- **Class C** — receiver continuously open. Near-instant downlink, mains power effectively required.
- **Spreading factor** trades range against airtime: SF12 reaches furthest but occupies the channel far longer. **ADR** (Adaptive Data Rate) tunes this per device.
- **Duty cycle limits** are a legal constraint in Europe (typically 1 % per channel in EU868). A device cannot simply transmit more often because you want fresher data. This is the single most common design mistake in LoRaWAN projects.
- **Security**: AES-128 with two distinct keys — **NwkSKey** for network-layer integrity and **AppSKey** for end-to-end payload encryption. The network operator cannot read payloads if key management is done properly. **OTAA** (over-the-air activation) is strongly preferable to **ABP**, which has replay weaknesses.
- Payloads are a few tens of bytes and **not self-describing**; a decoder per device type lives on the application server.

Good for: utility metering, tank levels, environmental monitoring, agriculture, asset presence over a wide campus, anything reporting a few times an hour for years on a battery.

Bad for: anything needing low-latency downlink, firmware updates of meaningful size, or frequent reporting.

NB-IoT and LTE-M

Licensed-spectrum cellular, operator-managed, no gateway infrastructure of your own.

- **NB-IoT** — very narrow band, excellent deep-indoor penetration, lowest power, but high latency (seconds) and no mobility handover. Best for static meters.
- **LTE-M** (eMTC) — higher rate, lower latency, supports **mobility and voice**, so the right pick for moving assets and anything interactive.
- Both use **PSM** (Power Saving Mode) and **eDRX** to sleep for hours while keeping their network registration, which is what delivers multi-year battery life.
- Management is typically **LwM2M over CoAP** (see 07).

The practical considerations are commercial rather than technical: carrier coverage at the exact installation point, module certification per region, roaming agreements for assets that cross borders, and the fact that 2G/3G sunsets have stranded a generation of older cellular devices. Verify coverage by survey, not by coverage map.

Private 5G and URLLC

5G URLLC targets 1 ms air latency with very high reliability, and **network slicing** allows an isolated virtual network for control traffic. With CBRS in the US or local licences elsewhere, a plant can run its own 5G network.

This is credible and deploying now for AGV fleets, large-area asset tracking, mobile robots and remote operation of mining and port equipment — applications where cables are impossible and Wi-Fi roaming is inadequate. It is expensive and requires genuine RF engineering, so it suits large sites with mobility problems rather than general sensor connectivity.

Coexistence: planning the 2.4 GHz band

WirelessHART, ISA100, Zigbee, BLE and Wi-Fi all share 2.4 GHz. Uncoordinated, they degrade each other.

Practical measures:

- Restrict Wi-Fi to channels 1, 6 and 11 and allocate the remaining spectrum to 802.15.4 channels that do not overlap (15, 20, 25, 26).
- Blacklist the channels used by plant Wi-Fi in the WirelessHART or ISA100 manager.
- Move high-bandwidth Wi-Fi to 5 GHz or 6 GHz entirely, leaving 2.4 GHz for sensors.

- Survey before and after installation; RF environments change when machinery, racking and liquid-filled vessels move.
- Remember that metal structures, dense pipework and full vessels attenuate heavily — a link that works on an empty tank may fail when it is full.

Pitfalls

- **Ignoring LoRaWAN duty cycle limits** and designing a reporting interval that is not legal in your region.
- **Choosing NB-IoT for a mobile asset.** It has no handover; the device drops and must re-register.
- **Specifying battery life from the datasheet headline.** Real life depends on report interval, spreading factor or data rate, retries, and temperature. Cold storage roughly halves many chemistries' usable capacity.
- **Deploying 2.4 GHz mesh without a Wi-Fi channel plan.**
- **Assuming a wireless mesh will self-heal around any failure.** Mesh needs redundant neighbours; a device at the end of a long chain has no alternative path, and WirelessHART managers will report poor path diversity if you look.
- **Forgetting antenna placement is engineering.** Mounting a radio inside a metal enclosure, or flat against a steel beam, loses most of the link budget regardless of protocol.
- **No plan for key provisioning at scale.** Join keys and device credentials for a thousand devices need a managed process, not a spreadsheet.

09 — Security

Why OT security is not IT security with different cables

The priorities invert. IT security classically ranks **confidentiality, integrity, availability**. OT ranks **safety, availability, integrity, confidentiality** — and safety sits above everything, because the failure mode is not a data breach but an injured person or a destroyed asset.

Practical consequences that surprise IT teams:

- **You often cannot patch.** A controller running a validated pharmaceutical process or a 20-year-old turbine governor may have no vendor patch, and changing it may require requalification costing more than the plant earns in a week.
- **You cannot reboot to apply updates.** A continuous process may run for 18 months between planned shutdowns.
- **Active scanning can cause outages.** Many embedded devices have fragile network stacks; an ordinary Nmap scan has halted PLCs. Use passive monitoring or OT-aware scanners with explicit vendor approval.
- **Availability beats confidentiality.** An authentication mechanism that locks out a legitimate operator during an emergency is a safety hazard, not a security control.
- **Lifetimes are decades.** Equipment specified today will still be running when the cryptography chosen for it is obsolete. Plan for algorithm replacement.

IEC 62443

IEC 62443 is the reference standard for industrial automation and control system security. Its structure is worth knowing because it provides the vocabulary used in procurement and audit.

Part group	Covers
62443-1-x	Concepts, terminology, models
62443-2-x	Policies and procedures — security programme, patch management, service providers
62443-3-x	System level — risk assessment, system design, security levels
62443-4-x	Component level — secure development lifecycle and technical requirements for products

Zones and conduits

The central design concept. Group assets with common security requirements into a **zone**; every communication path between zones is a **conduit** that is explicitly defined and controlled. There are no undeclared paths.

This is how you make a flat, unsegmented plant network tractable: enumerate zones, enumerate the conduits you actually need, then deny everything else.

Security Levels

SL	Protects against
SL 1	Casual or coincidental violation
SL 2	Intentional violation using simple means, low motivation
SL 3	Intentional violation using sophisticated means, moderate resources
SL 4	Intentional violation using sophisticated means, extended resources

Target SLs are assigned per zone from a risk assessment. Most manufacturing aims at SL 2, with SL 3 for safety-instrumented or high-consequence zones. Note that **SL 3-4 is not reachable with the native capability of most protocols in this collection** — it requires compensating architecture.

The Purdue model as a segmentation plan

Level	Contents	Trust
5	Enterprise network, internet	Untrusted
4	Business / site IT	Low
3.5	**DMZ** — historian replica, patch server, jump host, broker	Mediation boundary
3	Site operations: MES, central historian, engineering workstations	Medium
2	Supervisory: SCADA servers, HMIs	High
1	Control: PLCs, DCS controllers, safety systems	Critical
0	Field: sensors, actuators, drives	Critical

The rule that matters most: **no direct traffic between Level 4 and Levels 0-2.** Everything crosses through Level 3.5, where it can be brokered, inspected, logged and rate-limited.

In concrete terms, this is why the standard IIoT pattern is an **edge gateway in the DMZ** that reads from the control network and publishes outbound to the cloud, rather than a cloud service reaching into the plant. Outbound-only connections from a mediated zone are vastly easier to secure than inbound ones.

Protocol security posture at a glance

Protocol	AuthN	Integrity	Encryption	Verdict
Modbus RTU/TCP	None	CRC only (not cryptographic)	None	Segment it; never expose
PROFINET	None for cyclic data	None	None	Rely on segmentation
EtherNet/IP	Optional (CIP Security)	With CIP Security	With CIP Security	Enable if supported
EtherCAT	None	None	None	Physical security is the control
DNP3	SAv5 available	Yes with SAv5	TLS variants	Enable SAv5
BACnet	Weak historically	BACnet/SC adds TLS	BACnet/SC	Migrate to BACnet/SC
MQTT	TLS + credentials	TLS	TLS	Good when configured; ACLs essential
OPC UA	X.509 mutual + user tokens	Per-message signing	Per-message encryption	Strongest native model
CoAP	DTLS / OSCORE	Yes	Yes	Good; PSK management is the work
AMQP 1.0	SASL + TLS	TLS	TLS	Good
DDS	DDS Security plugins	Yes	Yes	Good but must be enabled
LoRaWAN	AES-128, OTAA	Yes	Yes (AppSKey end-to-end)	Good; use OTAA, not ABP
WirelessHART	AES-128, mandatory	Yes	Yes	Good by default

The pattern is clear: **control-layer protocols have essentially no native security, and IIoT-layer protocols have decent security that is frequently left switched off.**

TLS and PKI in an OT context

TLS is the workhorse, but industrial deployment has specific traps.

- **Certificate expiry is an availability risk.** An expired certificate stops communication exactly as effectively as a cut cable, and it will happen at an inconvenient moment. Inventory every certificate with its expiry date, alert 90 days ahead, and rehearse rotation. For OPC UA at scale, a GDS automates issuance and renewal.
- **Devices with no real-time clock cannot validate expiry.** A device that boots thinking it is 1970 will reject a valid certificate. Provide NTP, or accept the limitation deliberately.
- **Verify the chain and the hostname.** Disabling verification "temporarily during commissioning" is how most MQTT and OPC UA deployments end up with no meaningful authentication. If a device cannot validate properly, record it as an accepted risk rather than quietly turning the check off.
- **Prefer client certificates over passwords for fixed devices.** They cannot be shoulder-surfed, shared or reused across sites, and revocation is centralised.
- **Plan for crypto-agility.** Equipment installed now will outlive current algorithm recommendations; favour devices whose firmware can be updated and whose trust stores are manageable.

Secure remote access

Vendor remote access is both operationally necessary and the most common serious exposure. What works:

- Access brokered through a **jump host in the DMZ**, never a direct tunnel to a controller.
- **Multi-factor authentication** for every human.
- **Operator-approved, time-limited sessions** — access enabled on request for a window, then disabled. Ideally a physical key switch or an HMI control that an operator must actively grant.
- **Session recording** for audit and for understanding what changed.
- **No persistent vendor VPNs** with standing credentials and no monitoring. Cellular modems quietly fitted to machines by vendors are a recurring finding in OT assessments and bypass the entire network architecture.

Monitoring that does not break things

- **Passive network monitoring** via a SPAN or mirror port, or a network tap, is the default OT-safe approach: full visibility, zero injected traffic.
- **Protocol-aware deep packet inspection** gives real value here, because industrial protocols are predictable. A Modbus write function code to a device that has only ever been read, or a PROFINET DCP reset command, is a strong signal — far clearer than equivalent anomalies in IT traffic.
- **Asset inventory derived from passive observation** is often the single most useful early deliverable, because most sites do not know precisely what is connected.
- **Baseline the normal.** OT traffic is far more regular than IT traffic, which makes anomaly detection genuinely effective once a baseline exists.

A pragmatic priority order

If you are starting with a flat plant network and limited budget:

1. **Inventory.** Passively discover what is connected and what talks to what.
2. **Segment.** Separate OT from IT with a firewall and a DMZ. This single step removes more risk than everything else combined.
3. **Remove internet exposure.** Search your public IP ranges for exposed Modbus (502), OPC UA (4840), BACnet (47808), VNC and RDP. Close them.

4. **Control remote access.** Brokered, MFA, time-limited, recorded.
5. **Harden the IIoT path.** TLS everywhere, per-device credentials, default-deny broker ACLs, outbound-only from the DMZ.
6. **Back up and rehearse recovery.** Controller programs, HMI projects, network configuration, and a tested restore procedure. Ransomware recovery in OT depends almost entirely on this.
7. **Then** consider detection, patching programmes and SL uplift.

Pitfalls

- **Confusing functional safety with cybersecurity.** PROFIsafe, CIP Safety and FSoE protect against random faults and transmission corruption. They do not stop an attacker; a valid safety message with a valid CRC is accepted whoever sent it.
- **Relying on an air gap that does not exist.** USB sticks, engineering laptops, vendor modems and dual-homed historians are all paths. Verify, do not assume.
- **Allowing a dual-homed host to bridge zones.** A historian with one NIC in Level 3 and one in Level 4 is a conduit regardless of what the architecture diagram claims.
- **Leaving OPC UA `SecurityPolicy#None` enabled** after commissioning.
- **Shared credentials across a device fleet.** One extracted key compromises everything; rotation becomes impossible.
- **Active vulnerability scanning of production OT** without vendor sign-off and a maintenance window.
- **Writing a security policy that operators must bypass to do their jobs.** It will be bypassed, and you will have lost both the control and the visibility.

10 — Integration Patterns

The canonical IIoT data path

Almost every successful plant-to-cloud architecture is some version of this:

```
Field devices          Control            Edge (DMZ)              Platform
-----------------      ---------------    ------------------      ----------------
sensors, drives  --->  PLC / DCS   --->   gateway          --->   broker / cloud
HART, IO-Link          OPC UA server      - protocol xlate        historian
Modbus, PROFIBUS       Modbus TCP         - normalise             analytics
                                          - buffer                dashboards
                                          - publish outbound      MES / ERP
```

The important properties are structural, not technological:

1. **Read from the controller, not the field bus**, wherever possible. The PLC already polls the devices correctly, with the right scaling and the right engineering. A second master on the fieldbus adds load and risk.
2. **Normalise at the edge.** The gateway is where register 40107 becomes `Line3/Filler/Motor1/PowerFactor` with a unit and a timestamp. Do it once, close to the source, not in every consumer.
3. **Buffer at the edge.** The uplink will fail. Decide what happens to data during the outage before it happens.
4. **Publish outbound only.** The gateway initiates connections to the platform; nothing from outside initiates a connection into the plant. This is the single most valuable security property of the whole design.

Protocol conversion: what you lose at each hop

Every translation discards something. Know what, so you can decide whether it matters.

Conversion	What is lost
OPC UA to MQTT (raw JSON)	StatusCode quality, source timestamp, data types, the type hierarchy
OPC UA to Sparkplug B	The object model and references; metric names flatten to strings
Modbus to anything	Nothing is lost because nothing was there — but nothing is gained either; you must add units, scaling and names from a manual
PROFINET/EtherNet/IP to OPC UA	Cyclic timing context; you get values, not the deterministic guarantee
HART to Modbus	Device diagnostics, multiple variables, device identity
Anything to a flat tag list	Hierarchy, relationships, and the ability to discover

The two losses that cause most downstream trouble:

- **Quality and status.** If the sensor is faulty and OPC UA says `Bad_SensorFailure`, a gateway that forwards only the number turns a known failure into a believable reading. Analytics then trains on garbage. **Always carry quality through**, even if it means a slightly richer payload.
- **Source timestamps.** If the gateway stamps messages with its own clock on publication, you lose the ability to order events correctly and to tell a delayed batch from a current one. After an outage, a thousand buffered readings all arrive with the same gateway timestamp and the history is meaningless. **Preserve the original timestamp** and add the gateway time separately if you need it.

Edge buffering and store-and-forward

Design this explicitly rather than discovering the default behaviour during your first outage.

Decisions to make:

- **How long must you survive disconnected?** A cellular site might need 24-72 hours. Multiply by the message rate and payload size to size the store.
- **What is the storage medium?** RAM is fast but lost on power failure — which often accompanies network failure. SD cards wear out under continuous write; specify industrial-grade with wear levelling, or use a circular buffer sized well below capacity.
- **What happens when the buffer is full?** Drop oldest (keep recent trend), drop newest (keep the start of the event), or downsample older data progressively. All three are defensible; silence is not.
- **How fast do you drain?** Dumping 72 hours of backlog at full rate can saturate the uplink and overwhelm the ingest endpoint, which then throttles you, which looks like a new outage. Rate-limit the catch-up.
- **Will consumers cope with out-of-order and late data?** A dashboard showing "last value" will jump backwards in time during a drain. Downstream systems need to key on source timestamp, not arrival order.

Unified Namespace rollout

If you adopt the UNS pattern from 01, the namespace design is the most consequential decision in the project — it is the interface every future consumer codes against.

Designing the topic hierarchy

Make it mirror the **physical and organisational reality**, because that is what stays stable:

```
<enterprise>/<site>/<area>/<line>/<cell>/<device>/<metric>
```

Guidelines that prevent later pain:

- **Describe location and function, not technology.** `line3/filler/motor1` survives a PLC replacement; `plc_192_168_1_50/db100/dbw24` does not.
- **Be consistent about depth.** Consumers write wildcard subscriptions like `+/+/+/+/+/+/temperature`; irregular depth breaks them.
- **Lower case, no spaces, no national characters.** Topics are case-sensitive and some tooling handles Unicode poorly.
- **Never put values or units in topic names.** `temperature_celsius` forces a topic rename when you move to Fahrenheit; put the unit in the payload.
- **Separate current state from events and commands.** A common split is a state branch, an event branch and a command branch, so ACLs can grant publish rights to state without granting command rights.
- **Write it down and version it.** The namespace is an API; treat it like one.

Phasing

1. **One line, read-only.** Prove the pattern end to end on a single production line, publishing only. No writes, so the risk to production is near zero.
2. **Add consumers.** Dashboard, historian. Demonstrate that adding a consumer needs no change at the edge — this is the argument that wins support for the pattern.
3. **Extend horizontally** to more lines using the same namespace template.
4. **Add commands last**, with strict ACLs, confirmation semantics and an audit trail. Writing to a plant from a message broker deserves real care.

Which protocol for which hop

Hop	Default choice	Why
Sensor to controller	The machine's native fieldbus	Determinism; it is already there
Instrument diagnostics	HART-IP or IO-Link	Data already exists, cheap to harvest
Controller to edge	OPC UA	Semantics, security, vendor-neutral
Legacy device to edge	Modbus TCP via gateway	Universal, adequate for monitoring
Edge to platform	MQTT + Sparkplug B	Efficient, decoupled, self-describing, NAT-friendly
Platform to enterprise	AMQP or REST	Transactional semantics, existing IT tooling
Controller to controller, fast	OPC UA PubSub or native fieldbus	Deterministic one-to-many
Inside a robot or vehicle	DDS	QoS contracts, low latency, redundancy

Timestamping and time synchronisation

Industrial data is almost useless without trustworthy time, and this is routinely underestimated.

- **NTP** gives millisecond accuracy on a LAN — sufficient for SCADA, historians and analytics.
- **PTP / IEEE 1588** gives sub-microsecond accuracy with hardware support — needed for motion coordination, power quality and sequence-of-events analysis across devices.
- **Timestamp at the source** whenever the source has a clock. A reading stamped by the PLC is worth far more than one stamped by a cloud ingest endpoint three hops and an outage later.
- **Use UTC everywhere and convert for display only.** Local time in stored data creates an hour of duplicated or missing records twice a year, and the resulting analytics bugs are genuinely hard to find.
- **Keep one time source for the whole plant.** Two unsynchronised sources make cross-system correlation impossible, which defeats the main purpose of collecting the data.

Testing an integration before it reaches production

- **Read-only first.** Commission every path with writes disabled.
- **Verify against a known value.** Compare the value your pipeline reports against the HMI reading, for a quantity you can also confirm physically. Do this for one tag per device type before trusting hundreds.
- **Check scaling and endianness deliberately** (see 04). A plausible but wrong number is far more dangerous than an obviously broken one.
- **Test the failure modes, not just the happy path.** Pull the uplink cable. Power cycle the gateway mid-publish. Fill the buffer. Expire a certificate in a lab. These are the behaviours that determine whether the system is trustworthy.
- **Measure the load you add.** Check controller scan time and fieldbus cycle time before and after. If your monitoring extends the PLC scan, you have made the plant worse to observe it.
- **Confirm the quality path.** Force a sensor fault or disconnect a transmitter and verify that the downstream consumer shows bad quality rather than a stale number.

Pitfalls

- **Polling the same device from several systems.** Modbus TCP devices often allow only one or two connections. Aggregate at one gateway and fan out from there.
- **No decommissioning process.** Retired devices leave retained MQTT messages, stale OPC UA nodes and dead tags that quietly corrupt dashboards and models for years.

- **Mapping tables maintained by hand in spreadsheets.** This is the integration tax returning through the back door. Generate configuration from a single source of truth.
- **Gateway as a single point of failure with no monitoring.** Monitor the monitor: publish a heartbeat and alarm on its absence.
- **Letting each project invent its own namespace.** Three plants with three conventions cannot be analysed together, which is usually the entire business case.
- **Treating the edge gateway as appliance-like and never patching it.** It is a Linux computer in your DMZ with credentials to both sides. It needs the same lifecycle management as any server.

References — Official Specifications & Sources

These guides are originally written. This page tells you where to get the **normative documents** when you need the authoritative text — for implementing a stack, certifying a device, or settling an argument.

Why specifications are not mirrored in this repository

Most industrial protocol specifications are **copyrighted** and their licences do not permit redistribution. OPC Foundation, ODVA, PROFIBUS & PROFINET International, the EtherCAT Technology Group, IEC and ISO documents all fall into this category; several require membership or payment. Re-hosting them in a public repository would be copyright infringement, so this page links to the official sources instead.

Where a document genuinely is freely redistributable — IETF RFCs, for instance — the link below goes to the canonical source, which is more trustworthy than a copy anyway.

Access type	Meaning
Free	Downloadable at no cost, no registration
Registration	Free but requires an account
Member/paid	Requires membership or purchase

Core IIoT protocols

Protocol	Document	Body	Access	Where
MQTT 3.1.1	OASIS Standard	OASIS	Free	https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/
MQTT 5.0	OASIS Standard	OASIS	Free	https://docs.oasis-open.org/mqtt/mqtt/v5.0/
Sparkplug B	Sparkplug Specification 3.x	Eclipse Foundation	Free	https://sparkplug.eclipse.org/specification/
OPC UA	IEC 62541, all parts	OPC Foundation	Registration	https://reference.opcfoundation.org/
Modbus	Application Protocol Spec; Modbus over Serial Line	Modbus Organization	Free	https://www.modbus.org/specs.php
CoAP	RFC 7252	IETF	Free	https://www.rfc-editor.org/rfc/rfc7252
CoAP Observe	RFC 7641	IETF	Free	https://www.rfc-editor.org/rfc/rfc7641
CoAP Block-wise	RFC 7959	IETF	Free	https://www.rfc-editor.org/rfc/rfc7959
CoAP over TCP/TLS	RFC 8323	IETF	Free	https://www.rfc-editor.org/rfc/rfc8323
OSCORE	RFC 8613	IETF	Free	https://www.rfc-editor.org/rfc/rfc8613
CBOR	RFC 8949	IETF	Free	https://www.rfc-editor.org/rfc/rfc8949
SenML	RFC 8428	IETF	Free	https://www.rfc-editor.org/rfc/rfc8428
AMQP 1.0	OASIS Standard / ISO-IEC 19464	OASIS	Free	https://docs.oasis-open.org/amqp/core/v1.0/
DDS	DDS 1.4, DDSI-RTPS 2.5, IDL, DDS Security	OMG	Free	https://www.omg.org/spec/DDS/
LwM2M	OMA LwM2M	OMA SpecWorks	Free	https://openmobilealliance.org/specifications/lwm2m

The OPC Foundation's online reference is genuinely good and free to browse after registration — start there rather than buying the IEC version unless you need the IEC document for formal reasons.

Industrial Ethernet & fieldbus

Protocol	Body	Access	Where
PROFINET / PROFIBUS / PROFIsafe	PI (PROFIBUS & PROFINET International)	Free profiles, member specs	https://www.profibus.com/technology/profinet
EtherNet/IP, CIP, CIP Safety, CIP Security	ODVA	Member/paid	https://www.odva.org/technology-standards/
EtherCAT	EtherCAT Technology Group	Membership (free for most users)	https://www.ethercat.org/
Ethernet POWERLINK	EPSG	Free	https://www.ethernet-powerlink.org/
Sercos III	Sercos International	Free overviews	https://www.sercos.org/
IO-Link	IO-Link Community (within PI)	Free	https://io-link.com/en/Documents/
HART / WirelessHART	FieldComm Group	Registration/member	https://www.fieldcommgroup.org/
Foundation Fieldbus	FieldComm Group	Member	https://www.fieldcommgroup.org/
CANopen	CAN in Automation (CiA)	Free for CiA 301 after registration	https://www.can-cia.org/standardization/specifications/
SAE J1939	SAE International	Paid	https://www.sae.org/standards/
AS-Interface	AS-International	Free overviews	https://www.as-interface.net/
Ethernet-APL	APL project (PI, ODVA, FieldComm, OPC-F)	Free	https://www.ethernet-apl.org/

Building, utility & substation

Protocol	Body	Access	Where
BACnet	ASHRAE / BACnet International	Paid (ASHRAE 135)	https://www.ashrae.org/technical-resources/bookstore
BACnet overview and resources	BACnet International	Free	http://www.bacnet.org/
DNP3	IEEE 1815 / DNP Users Group	Paid	https://www.dnp.org/
IEC 60870-5	IEC	Paid	https://webstore.iec.ch/
IEC 61850	IEC	Paid	https://webstore.iec.ch/

Wireless

Technology	Body	Access	Where
LoRaWAN	LoRa Alliance	Free	https://resources.lora-alliance.org/technical-specifications
Zigbee / Matter / Thread	Connectivity Standards Alliance	Free	https://csa-iot.org/all-solutions/
Thread	Thread Group	Registration	https://www.threadgroup.org/support#specifications
Bluetooth / BLE	Bluetooth SIG	Free	https://www.bluetooth.com/specifications/specs/
IEEE 802.15.4	IEEE	Paid (some via GET programme)	https://standards.ieee.org/
NB-IoT / LTE-M	3GPP	Free	https://www.3gpp.org/specifications
WirelessHART (IEC 62591)	IEC / FieldComm	Paid / member	https://webstore.iec.ch/
ISA100.11a (IEC 62734)	ISA / IEC	Paid	https://www.isa.org/standards-and-publications

Time-Sensitive Networking

Standard	Topic	Where
IEEE 802.1AS	Timing and synchronisation (gPTP)	https://standards.ieee.org/
IEEE 802.1Qbv	Time-aware shaper	https://standards.ieee.org/
IEEE 802.1Qbu / 802.3br	Frame pre-emption	https://standards.ieee.org/
IEEE 802.1CB	Frame replication and elimination	https://standards.ieee.org/
IEEE 1588	Precision Time Protocol	https://standards.ieee.org/

Many IEEE 802 standards are available at no cost through the **IEEE GET Program** — check there before purchasing: https://standards.ieee.org/products-programs/ieee-get-program/

Security

Document	Topic	Body	Access	Where
IEC 62443 (all parts)	Industrial automation security	IEC / ISA	Paid	https://webstore.iec.ch/
ISA/IEC 62443 overview	Free introductory material	ISA	Free	https://www.isa.org/standards-and-publications/isa-standards/isa-iec-62443-series-of-standards
NIST SP 800-82 Rev 3	Guide to OT Security	NIST	**Free**	https://csrc.nist.gov/pubs/sp/800/82/r3/final
NIST Cybersecurity Framework 2.0	Risk management framework	NIST	**Free**	https://www.nist.gov/cyberframework
NIST SP 800-52 Rev 2	TLS configuration guidance	NIST	**Free**	https://csrc.nist.gov/pubs/sp/800/52/r2/final
CISA ICS advisories	Live vulnerability advisories	CISA	**Free**	https://www.cisa.gov/news-events/cybersecurity-advisories
MITRE ATT&CK for ICS	Adversary techniques in ICS	MITRE	**Free**	https://attack.mitre.org/matrices/ics/

NIST SP 800-82 is the single most useful free document in this table. If you read only one external reference on OT security, read that one.

Open-source implementations worth studying

Reading a good implementation often teaches more than the specification.

Project	Protocol	Notes
Eclipse Mosquitto	MQTT	Small, readable C broker
Eclipse Paho	MQTT	Client libraries in many languages
EMQX / VerneMQ / HiveMQ CE	MQTT	Clustered brokers
open62541	OPC UA	C99, permissive licence, widely used in embedded work
Eclipse Milo	OPC UA	Java client and server
node-opcua	OPC UA	Node.js, good for prototyping
libmodbus	Modbus	The reference C library
pymodbus	Modbus	Python client and server
Eclipse California	CoAP	Java CoAP and DTLS
libcoap	CoAP	C implementation
Eclipse Cyclone DDS	DDS	Permissive licence, used by ROS 2
eProsima Fast DDS	DDS	Default ROS 2 middleware
SOEM / SOES	EtherCAT	Simple Open EtherCAT Master and Slave
Telegraf	Many	Input plugins for Modbus, MQTT, OPC UA — fast way to prototype a pipeline

Tools for diagnosis

Tool	Use
Wireshark	Dissectors for Modbus, PROFINET, EtherCAT, EtherNet/IP, MQTT, DNP3, BACnet and more
UaExpert	The de facto OPC UA client for browsing and testing
MQTT Explorer / MQTTX	Topic tree visualisation and manual publish/subscribe
modpoll / mbpoll	Command-line Modbus clients for quick verification
Sparkplug-aware clients (e.g. Ignition, Chariot)	Decoding protobuf Sparkplug payloads
nmap with caution	Discovery — **never** against production OT without approval

A note on standards versions

Protocol specifications are revised. Before relying on a detail in these guides for implementation work, check the current version of the relevant document at the links above — particularly for OPC UA companion specifications, Sparkplug (which moved to 3.0 under Eclipse), and TSN profiles, all of which are actively evolving.